Tutorial -5

Course code - Name	U24AI403/ U24DS403 SOFTWARE ENGINEERING	**Branch- section**	4CSM/ 4CSD
Topics covered	The design Model, Creating an architectural design – Software architecture, Architectural styles and patterns, Architectural design, Assessing alternative architectural designs		

Tutorial	Problem No.	Tutorial Problems	CO	CDLL
T-5	1.	**A healthcare analytics system is being developed to predict patient risks using hospital data.** **a. Explain the purpose of the design model in structuring a system. [5 M - U]** **b. Construct the architecture of this system using a layered or modular model. [5 M - Ap]**	**CO2**	**[Ap]**
	Sol:	a. The **design model** acts as a blueprint that transforms requirements into a clear technical structure for implementation. Its main purposes are: 1. **Provides System Structure** o Defines how system components are organized and how they interact. o Breaks a complex healthcare analytics system into manageable modules such as data collection, analytics, and reporting. 2. **Bridges Requirements and Implementation** o Converts user needs (risk prediction, patient monitoring) into technical solutions like algorithms, databases, and interfaces. 3. **Improves Modularity and Maintainability** o Separates responsibilities into layers or components, making updates (e.g., new ML models) easier without affecting the entire system. 4. **Enhances Communication Among Stakeholders** o Developers, designers, and hospital staff can understand the system structure through diagrams and models. 5. **Supports Quality Attributes** o Helps achieve scalability (more hospital data), reliability (accurate predictions), and security (patient data protection). b. **Proposed Layered Architecture** **1. Presentation Layer (User Interface)**		

- Dashboards for doctors and hospital admins
- Risk alerts and patient reports
- Data visualization tools

2. Application/Service Layer (Business Logic)

- Patient risk evaluation rules
- Data validation and processing
- Workflow management (alerts, notifications)

3. Analytics Layer (AI/ML Processing)

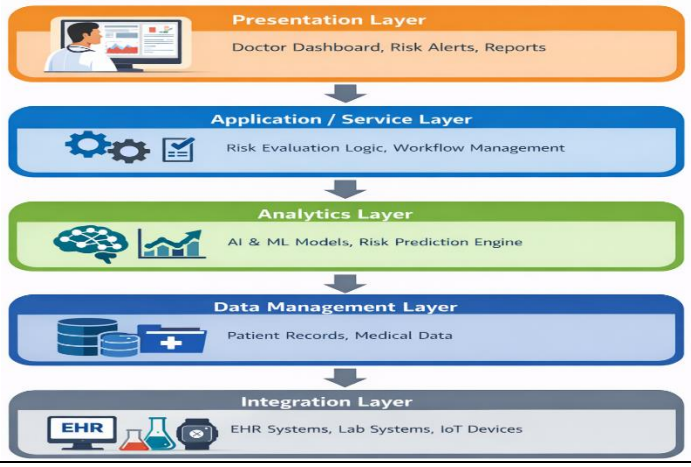
- Predictive models for disease risk
- Data mining and statistical analysis
- Model training and prediction engine

4. Data Management Layer

- Patient records database
- Medical history and hospital datasets
- Data cleaning and preprocessing modules

5. Integration Layer (External Systems)

- Electronic Health Records (EHR) systems
- Laboratory systems and wearable devices
- APIs for hospital information systems



2.	**An e-learning platform must scale to millions of users.** **a. Interpret the main elements of architectural design. [3 M - U]** **b. Make use of suitable architectural patterns to design a scalable platform. [4 M – Ap]** **c. Distinguish the layered and microservices styles in terms of scalability and maintenance. [3 M – An]**	**CO2**	**[An]**
Sol:	a. 1. **Components (Modules/Services)** • Independent functional units such as user management, course delivery, payment, and assessment modules.		

- Each component performs a specific responsibility in the system.

2. **Connectors (Communication Mechanisms)**
   - Define how components interact (APIs, message queues, REST services).
   - Enable data flow between frontend, backend, and databases.

3. **Configuration (System Structure)**
   - Arrangement and relationships among components.
   - Determines whether the system uses layered, client–server, or microservices architecture.

b. **Suggested Pattern: Microservices + Cloud-Based Architecture**

1. Presentation Service
- Web/mobile interfaces for students and instructors
- Content streaming and dashboards

2. API Gateway
- Single entry point for all client requests
- Load balancing and authentication

3. Core Microservices
- User Management Service
- Course Management Service
- Video Streaming Service
- Assessment & Quiz Service
- Payment & Subscription Service
- Notification Service

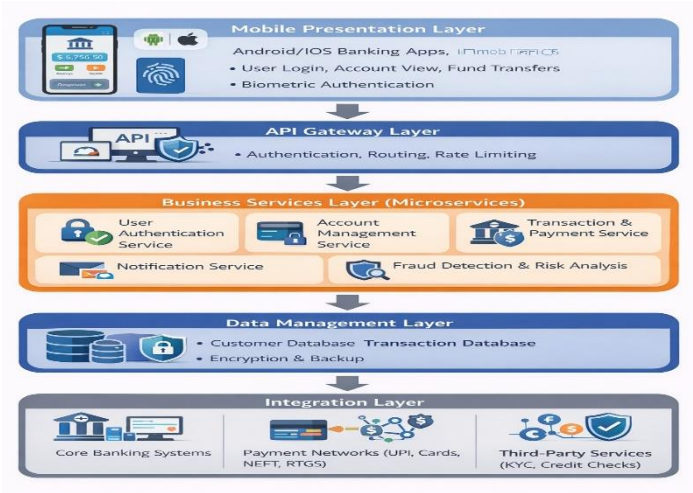
4. Data Layer
- Separate databases for each service (NoSQL/SQL)
- Distributed storage for videos and course materials

5. Infrastructure Layer
- Cloud hosting (auto-scaling servers)
- CDN for content delivery
- Message queues for asynchronous processing

c. Differences between layered and microservices styles

Aspect	Layered Architecture	Microservices Architecture
Scalability	Limited – scaling entire layers may be required	High – individual services scale independently
Maintenance	Easier initially but becomes complex as system grows	Easier long-term due to independent services
Deployment	Usually deployed as a single application	Each service deployed separately

	Flexibility	Less flexible for large distributed systems	Highly flexible for large-scale platforms
3.	**A banking application is being modernized for mobile-first services.** **a. Summarize what architectural styles are with relevant examples. [5 M – U]** **b. Build a high-level architecture suitable for mobile banking. [5 M – Ap]**	**CO2**	**[Ap]**
Sol:	a. **Architectural styles** are predefined ways of organizing software components and their interactions. They provide standard structures that guide system design and improve scalability, maintainability, and performance. **Common Architectural Styles:** 1. **Layered Architecture** o System divided into layers such as presentation, business logic, and data. o *Example:* Traditional banking systems with UI → services → database layers. 2. **Client–Server Architecture** o Clients request services from centralized servers. o *Example:* Mobile banking app (client) interacting with bank servers. 3. **Microservices Architecture** o Application split into small independent services communicating via APIs. o *Example:* Separate services for authentication, payments, notifications. 4. **Event-Driven Architecture** o Components communicate using events or messages. o *Example:* Real-time transaction alerts sent after a payment event. 5. **Service-Oriented Architecture (SOA)** o Reusable services shared across systems. o *Example:* Shared customer verification or payment gateway services. **b. Proposed Mobile-First Banking Architecture** **1. Mobile Presentation Layer** • Android/iOS banking apps • User login, account view, fund transfers • Biometric authentication interface **2. API Gateway Layer** • Handles client requests		

	• Authentication and rate limiting • Routes requests to backend services **3. Business Services Layer (Microservices)** • User Authentication Service • Account Management Service • Transaction & Payment Service • Notification Service (SMS/Email alerts) • Fraud Detection & Risk Analysis Service **4. Data Management Layer** • Customer database • Transaction database • Secure encryption and backup systems **5. Integration Layer** • Core banking systems • Payment networks (UPI, cards, NEFT/RTGS) • Third-party verification (KYC services) 		
4.	**A smart city traffic management system will optimize vehicle flow using IoT sensors.** **a. Outline the steps in developing an architectural design. [3 M – U]** **b. Utilize the publish–subscribe pattern and explain how it can be applied for traffic data processing. [4 M – Ap]** **c. Categorize the role of design patterns in addressing system challenges. [3 M – An]**	**CO2**	**[An]**
Sol:	a. 1. Analyze Requirements and Constraints • Identify system goals such as real-time traffic monitoring, congestion control, and scalability. • Consider constraints like IoT device compatibility, network latency, and data security. 2. Identify Components and Architecture Style • Decide major modules (sensor data collection, analytics engine, traffic control system, dashboards). • Choose suitable styles such as event-driven or publish–subscribe for		

	real-time processing.
	3. Define Interactions and Deployment
	• Specify communication methods (APIs, message brokers).
	• Map components to infrastructure like cloud servers, edge devices, and control centers.
	
	b. **Publish–Subscribe Pattern:**
	• Publishers send messages (events) without knowing receivers.
	• Subscribers receive only relevant data through topics/channels.
	• A message broker manages communication.
	Application in Smart Traffic System:
	• **Publishers:**
	  ○ IoT traffic sensors, cameras, GPS devices from vehicles
	  ○ Publish events such as vehicle count, speed, accidents.
	• **Message Broker (Middleware):**
	  ○ Handles event distribution (e.g., Kafka, MQTT broker).
	  ○ Ensures scalability and real-time data delivery.
	• **Subscribers:**
	  ○ Traffic Analytics Engine → predicts congestion
	  ○ Traffic Signal Controller → adjusts signal timing
	  ○ Emergency Services → responds to accidents
	  ○ City Dashboard → visualizes traffic flow
	c. 1. **Improve Scalability and Flexibility**
	Patterns like publish–subscribe or microservices help manage increasing traffic data volumes.
	2. Enhance Maintainability and Reusability
	Standard solutions reduce complexity and make modules easier to modify or reuse.
	3. Support Reliability and Real-Time Processing
	Event-driven patterns enable asynchronous communication and fault tolerance.		

5.	**A cloud video-streaming platform must ensure low latency and handle high traffic.** **a. Explain the criteria for evaluating alternative architectural designs. [5 M – U]** **b. Apply a structured evaluation (e.g., ATAM) to recommend an architecture for this platform. [5 M – Ap]**	**CO2**	**[Ap]**
Sol:	a. When comparing different architectures for a cloud video-streaming platform, designers evaluate them based on key quality attributes and system needs: 1. **Performance and Latency** ○ Ability to deliver videos with minimal buffering and fast response time.		

2. **Scalability**
    - Capacity to handle millions of concurrent viewers by adding servers or resources dynamically.
3. **Reliability and Availability**
    - System uptime during peak traffic; fault tolerance and quick recovery from failures.
4. **Security and Data Protection**
    - Secure content delivery, user authentication, DRM protection, and privacy safeguards.
5. **Maintainability and Flexibility**
    - Ease of updating streaming features, adding new services, or modifying components without disrupting users.

b. **Step 1: Identify Business Drivers**

- High-quality streaming with low latency
- Massive user traffic handling
- Global accessibility
- Cost-efficient cloud operations

Step 2: Define Quality Attribute Scenarios

- Thousands of users join a live stream simultaneously.
- Users expect HD streaming with minimal buffering.
- System must continue functioning even if a server fails.

Step 3: Propose Candidate Architecture
Microservices + CDN-based Cloud Architecture

- API Gateway
- Video Ingestion & Encoding Service
- Streaming Delivery Service
- Recommendation Service
- User Management & Authentication
- Distributed storage + Content Delivery Network (CDN)

Step 4: Analyze Architectural Approaches & Tradeoffs

- Microservices improve scalability but increase deployment complexity.
- CDN reduces latency but adds infrastructure cost.
- Auto-scaling improves performance but requires monitoring mechanisms.

Step 5: Evaluate Risks & Recommend Final Architecture

- Recommended Architecture:
    - Microservices deployed on cloud containers
    - CDN for global content delivery
    - Load balancers and caching layers

		○ Event-driven messaging for streaming analytics		
	6.	**A ride-sharing system must be designed for real-time ride allocation and tracking.** **a. Summarize the software architecture in the context of system quality. [3 M – U]** **b. Develop an event-driven architecture for ride allocation and tracking. [4 M – Ap]** **c. Analyse how architectural choices affect performance, reliability, and security. [3 M – An]**	**CO2**	**[An]**
	Sol:	a. Software architecture defines the **high-level structure** of a system and strongly influences its quality attributes: 1. **Performance** – Efficient component interaction ensures quick ride matching and real-time updates. 2. **Reliability** – Fault-tolerant architecture ensures rides continue even if a service fails. 3. **Scalability & Maintainability** – Modular design allows the system to handle many users and simplifies updates. b. **Key Idea:** Components communicate using events instead of direct calls. **Main Components** • **Mobile Apps** – Rider and driver apps publish ride requests and location updates. • **Event Broker** – Message queue (Kafka/RabbitMQ) distributes events. • **Ride Matching Service** – Assigns nearest driver. • **Location Tracking Service** – Monitores driver movement. • **Notification Service** – Sends ride status alerts. • **Payment Service** – Processes fare transactions. **Event Flow** 1. Rider sends **Ride Request Event**. 2. Broker forwards event to **Ride Matching Service**. 3. Driver accepts → **Ride Accepted Event** generated. 4. **Location Updates** sent continuously during the ride. 5. Ride completion triggers **Payment Event** and notification updates. c **Quality Attribute** — **Architectural Impact** **Performance** — Event-driven and microservices allow parallel processing and faster response times. **Reliability** — Decoupled services prevent single-point failures and support fault tolerance. **Security** — API gateways, encrypted messaging, and authentication services protect sensitive ride and payment data.		